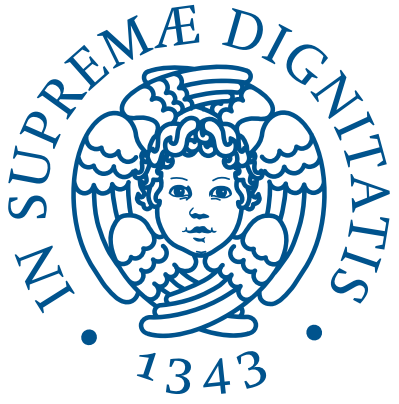


UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Parallel and Distributed Systems: Paradigms and Models

Distributed ad-hoc compressor/decompressor using Miniz

Lecturers:

Massimo Torquati

Marco Danelutto

Student:

Giuliano Difranco

2023/2024

1 Introduction

This project consist in implementing a compressor/decompressor using Miniz, a library that performs lossless compression using the zlib (RFC 1950) and Deflate (RFC 1951) compressed data format specification standards. The compressor/decompressor has to be implemented in two version, a parallel version and a distributed one. The parallel version is implemented using Fast Flow while the distributed one has to use MPI.

2 Structure of the compressed file

The structure of compressed file has been changed to allow the compression to be parallelized. At the beginning the input file is divided in N blocks of equal size and each block is compressed using Minizip. An header is created to store in how many blocks the original file has been split and the length of each block. The header of the compressed file is shown in Figure 1, the size of the header is $8(N+2)$ bytes where N is the number of blocks the uncompressed file has been split. The first 16 bytes are always used to store the size of the uncompressed file and the number of blocks. The remaining bytes of the header are used to store the length of each compressed block.

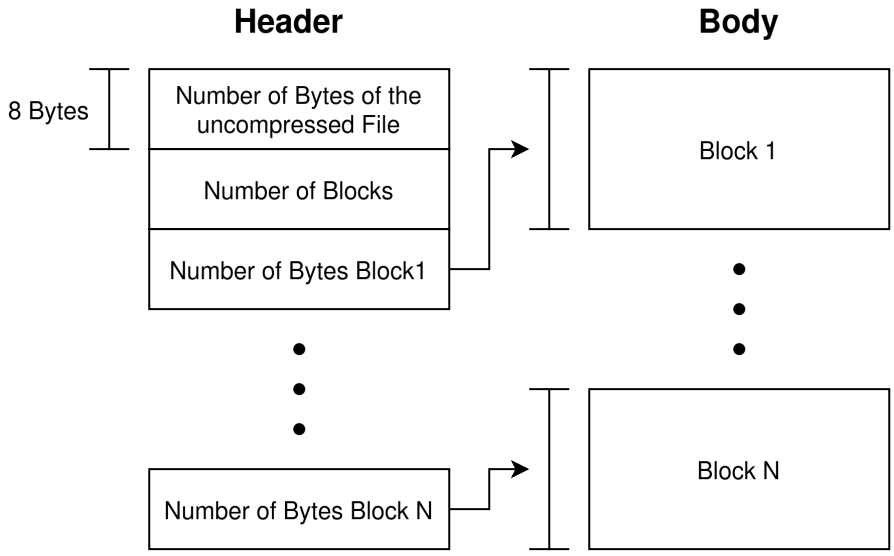


Figure 1: Structure of the compressed file. In the header each square is a size_t of 8 bytes. The first 8 bytes are used to store the size of the uncompressed file and 8 more bytes to describe in how many blocks the file has been split. The remaining bytes of the header are used to store the size of each compressed block. In the body the size of each block isn't fixed and it is stored in the header.

3 Sequential

The sequential code has been modified to match the FastFlow and MPI implementation, in this way a compressed file can be decompressed by all three version. From command line the user has to give two input parameters, c for compressing or d for decompressing and the path of the file or directory containing the files.

3.1 Compressing

Initially the file is read and it is divided in N blocks, to calculate N the following formula is used:

$$N = \lceil S/M \rceil$$

where S is the size of the file and M is the size of the blocks, M is stored in the variable $BIGFILE\_LOW\_THRESHOLD$. Each block is compressed using the function *mz_compress()* of Miniz and the length of each block is stored in the header. The header is attached to compressed blocks and the file is written.

3.2 Decompressing

The first 8 bytes of the compressed file are read and the exact size of the uncompressed file is allocated. The N blocks are then decompressed using the information stored in the header and the function *mz_decompress()* of Miniz. Then the file is written.

4 Fast Flow

The compressor/decompressor has been implemented using a All to All, the structure is described in Figure 2. Using a Multi-Input Node and a Multi-Output node and combining them together both the Left and Right Workers are able to receive or send from/to multiple nodes. From command line the user has to give four input parameters, c for compressing or d for decompressing, the path of the file or directory containing the files, the number L of Left Workers and the number R of Right Workers.

4.1 Left Worker

Initially, the folder is read and the F files are assigned equally to each Left Worker, at worst one or more LW can have one more file to read than the rest.

The behaviour of the Left Worker is described in the Algorithm 1, at the beginning of the execution of the program the Task in input is $NULL$ so the Left Worker read the files they are assigned to it and splits the data in blocks and send them to the Right Workers in Round Robin using the Fast Flow method *send_out*. If the task is not null then LW collects the blocks, when all the blocks of a file are collected the LW who collected the last block writes the file.

The collecting of the blocks is a concurrent task since more LW can update the variable, for this reason the F variables to count for the received blocks (one for each file) are atomic.

Algorithm 1 Left Worker

```
1: Input: File, Task
2: if Task == NULL then
3:     f ← ReadFile(File)
4:     blocks ← SplitFileIntoBlocks(f)
5:     SendBlocksToWorkers(blocks)
6: else
7:     N ← CollectBlocks(Task)
8:     if N == Task.FileNumberOfBlocks then
9:         WriteFile(Task.File)
10:    end if
11: end if
```

4.2 Right Worker

The Right worker receives the Task in input and compresses or decompresses the data. In case of compressing an array of *char* is created and sent to the Left Workers with the method *send_out*. For the decompression an array of *char* was created by the Left Workers for each file (since we know the uncompressed size by the header) so the Right worker simply decompresses the block and writes the data with a certain offset in the array. Finally it sends a task to one of the Left Workers to let it know its block has been uncompressed.

4.3 Performance

The performance of the Fast Flow implementations has been tested with two datasets, one consisting in only one file of dimension 2 GB and another one of 100 files of 20 MB each. Both datasets have been tested in a single node of the *spmcluster.unipi.it* which has 16 real cores and 32 logical cores.[1] The Figure 3a shows results of the compression of a 2 GB file, as expected the optimal number of Left Workers is one (since we have only one file) and the optimal number of Right Workers is 31 (since we have 32 logical cores). The same results happens for the decompression in Figure 3b. With the dataset of 100 files there isn't an improvement by adding more Left Workers in the compression (Figure 4a) but there is a clear improvement for the decompression as shown in the Figure 4b since the slowest part is the I/O for the decompression. The highest speedup is archived by the decompression of the 100 Files of 20MB (Figure 5a). In Figure 5b it is interesting to notice how quickly the efficiency drops as soon as we use more than 16 threads because we have 16 real cores. The weak scaling analysis

[1] A temporary folder is created in the node to store the files.

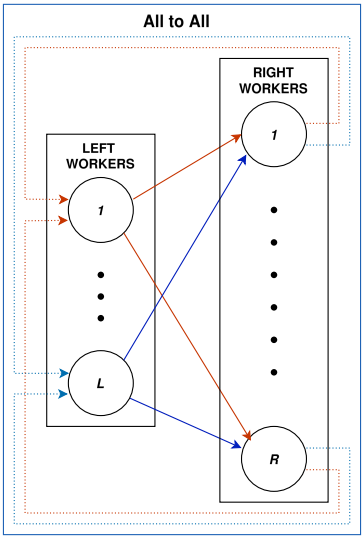


Figure 2: Implementation of the All to All in FastFlow. The files are read by the Left Workers, the data is split in blocks and sent to the Right Workers. Each block after being compressed/decompressed by the Right Workers is sent back to the Left workers using the feedback channels. When all blocks of file are received the file is written to disk by one of the Left Workers.

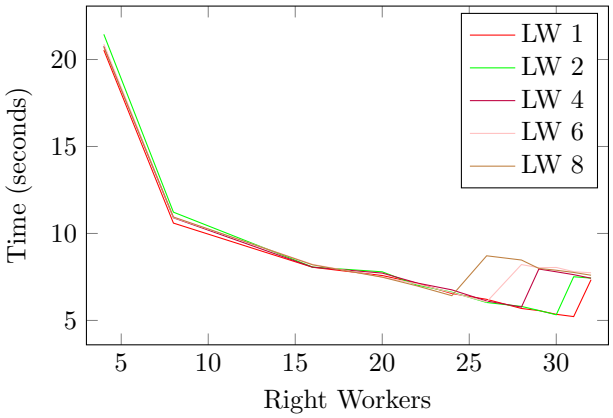
has been done using a dataset of 50MB * T where T is the number of threads. The implementation has a good weak scalability (Figure 6b) and speedup and efficiency are very similar to the strong analysis (Figure 7a and Figure 7b).

5 MPI

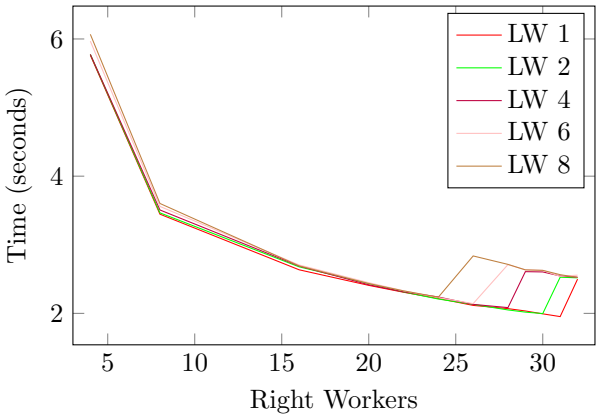
The MPI version has been implemented as a Master-slave architecture where the node with ID 0 is the master and the $N-1$ are the slaves. This implementation tries to use all the threads available in each Node of the cluster using *Fast Flow* for the slaves and *OpenMultiProcessing* for the master.

5.1 Master

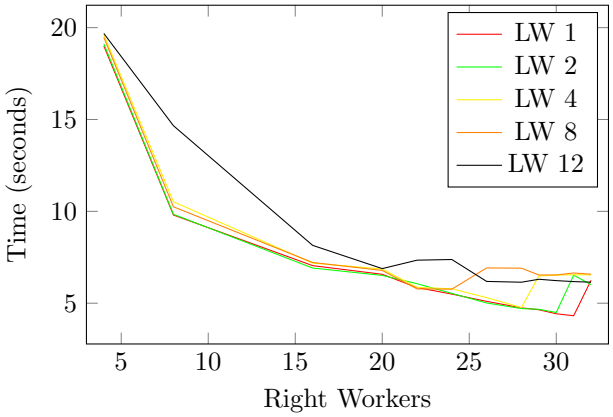
Initially the folder containing the files is read by the master, it sends to each worker the sizes of the files they have to process. In this way each worker can get an estimation of the buffer they have to allocate for each file. Then, the Master using *parallel for* of OMP will use threads to read as many files in parallel as possible reducing the I/O time.



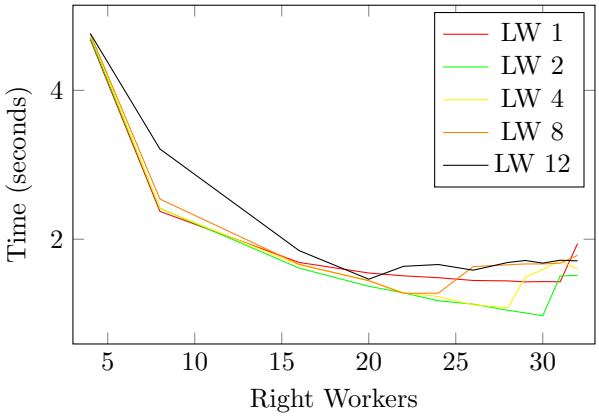
(a) Fast Flow: Time taken in seconds to **compress** a file of 2GB. Each colored line has different number of Left Workers. The best time is with 1 Left Worker and 31 Right Workers



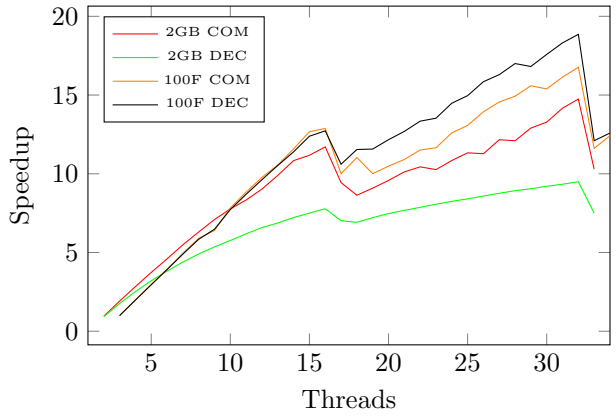
(b) Fast Flow: Time taken in seconds to **decompress** a file of 2GB. Each colored line has different number of Left Workers. The best time is with 1 Left Worker and 31 Right Workers



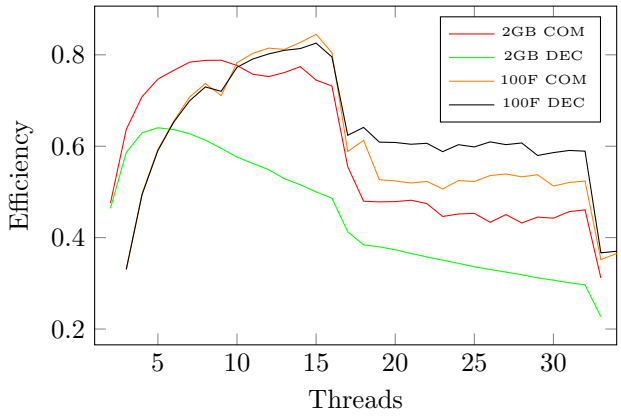
(a) Fast Flow: Time taken in seconds to **compress** a 100 files of 20MB. Each colored line has different number of Left Workers. The best time is with 1 Left Worker and 31 Right Workers.



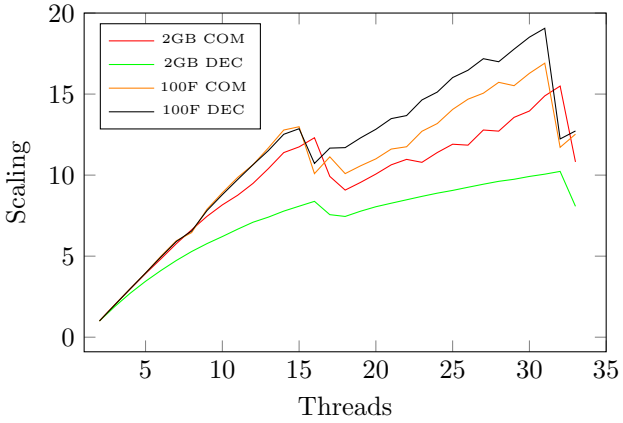
(b) Fast Flow: Time taken in seconds to **decompress** a 100 files of 20MB. Each colored line has different number of Left Workers. The best time is with 2 Left Worker and 30 Right Workers.



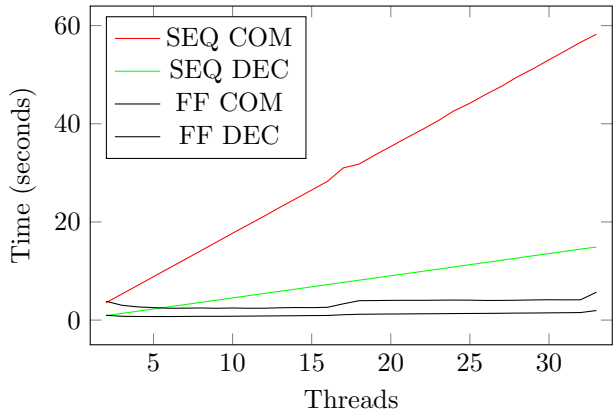
(a) Fast Flow: Speedup of the 2 datasets. The red and green line is the speedup of the compression and decompression of the 2GB dataset. The orange and purple line is the speedup of the compression and decompression of the 100 Files of 20 MB dataset.



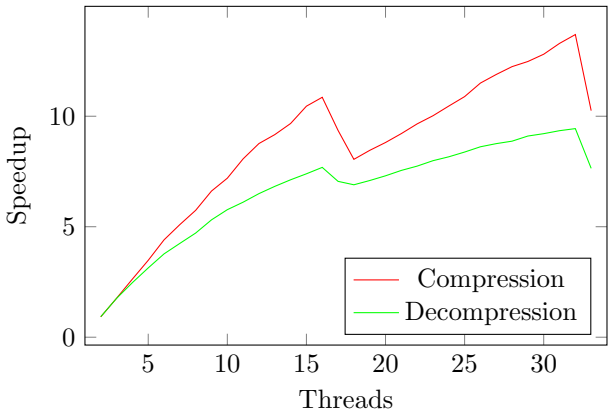
(b) Fast Flow: Efficiency of 2 datasets. The red and green line is the efficiency of the compression and decompression of the 2GB dataset. The orange and purple line is the efficiency of the compression and decompression of the 100 Files of 20 MB dataset.



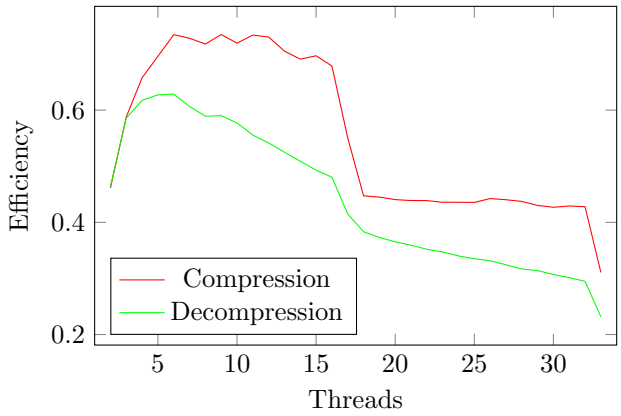
(a) Fast Flow: Scaling. T(1)/T(P) in this case is T(2) since it is mandatory to have at least 1 Left Worker and 1 Right Worker.



(b) Fast Flow: Weak Scaling Time. The dataset is one file of 50MB * T where T is the number of Threads. The left worker is always one while the right workers increase with the number of threads.



(a) Fast Flow: Weak Scaling Speedup. The dataset is one file of 50MB * T where T is the number of Threads. The left worker is always one while the right workers increase with the number of threads.



(b) Fast Flow: Weak Scaling Efficiency. The dataset is one file of 50MB * T where T is the number of Threads. The left worker is always one while the right workers increase with the number of threads.

5.1.1 Compressing

After the files are mapped the Master sends to each worker $N/NumberOfWorkers$ blocks in a single MPI_send for each file. To distinguish one file from the others the MPI_TAG has been used to give a different ID to each file. After all messages have been sent the Master will wait to have all the blocks and finally it will write the file.

5.1.2 Decompressing

Before sending the $N/NumberOfWorkers$ blocks to the workers, for the decompression the Master first sends the part of the header containing the sizes of the blocks. After, it sends the $N/NumberOfWorkers$ blocks to each worker. When all the data has been processed by the workers it is collect by the master and the file is written.

5.2 Slaves

Every process beside the Master implements an all to all with one Left Worker, R Workers and one Gatherer. The Left Worker receives MPI messages and passes the data to the workers, the workers process the data and send it to the Gatherer. The gatherer is the only one who sends messages. The architecture of the slave has been drawn in Figure 8. The All to All is attached to the Gatherer using a FastFlow Pipe.

5.2.1 Compressing

The Left Worker waits for messages and with an MPI_Probe reads the tag of the message before assigning the dimension to the buffer. The tag of the message contains the ID of the file, in this way the Left Worker can get an estimation of the dimension of the buffer. After receiving the data it is split into N blocks that are sent to the workers.

The worker compresses the blocks and send them to the Gatherer.

The Gatherer collects all the N blocks and attach an header to specify the length of each compressed block and sends them to the master.

5.2.2 Decompressing

The Left Worker collects two types of messages, the first one specifies the number of blocks to be received with each length. The second one is the actual data to be decompressed.

The behaviour of the workers and the Gatherer is the same for the decompression.

5.3 Performace

The evaluation of the performance has been tested using a big file of 2GB and 100 files of 20 MB each. In Figure 9a it is possible to see that using all 8 nodes and 15 Workers for each node the time of MPI is lower than the parallel version. While in Figure 9b the performance is way worse. The reason why this happens is because with a big file the Master sends to each Slave a single message containing more than one block, in this way the penalty of communication through MPI is eased. While with 100 files there are a lot of MPI messages for each file but the message has a smaller size and that worsen the performance. The speedup is higher than Fast Flow for the compression of a 2GB file but it is worse for everything else (Figure 10). The reason why the speedup is especially worse for the decompression is because we add extra cost on top of the reading and writing of the file that is the slowest part for the decompression. Also the decompression has an intermediate step that slows further the performance as explained in the subsection 5.2.2. We simplify the calculations we assumed we used all 32 Threads to measure the efficiency in Figure 11 so it was expected to have a very low efficiency. The weak scalability has been measured by increasing the dimension of a 200MB file by 200MB every Node was added. The implementation has a good weak scalability (Figure 12 but still the efficiency is quite low.

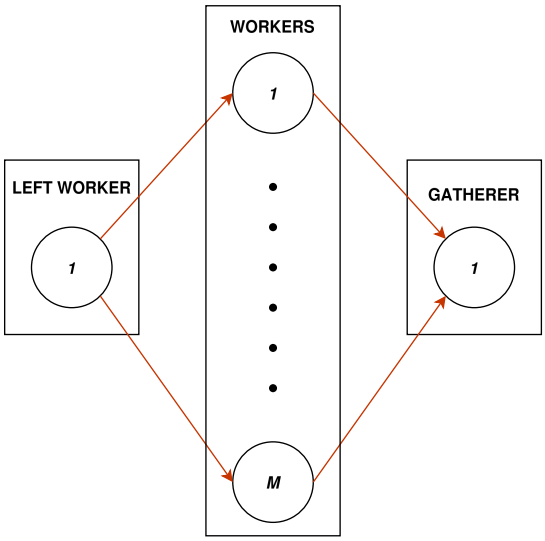
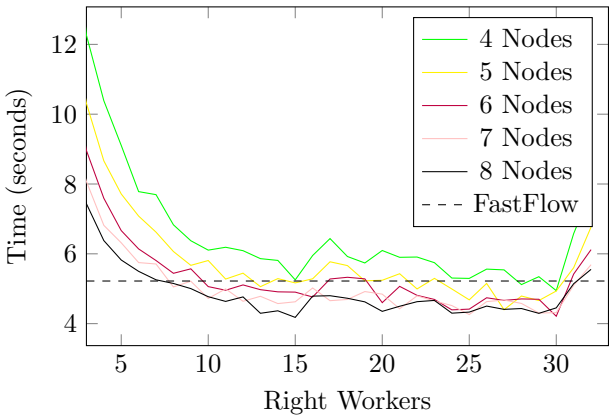
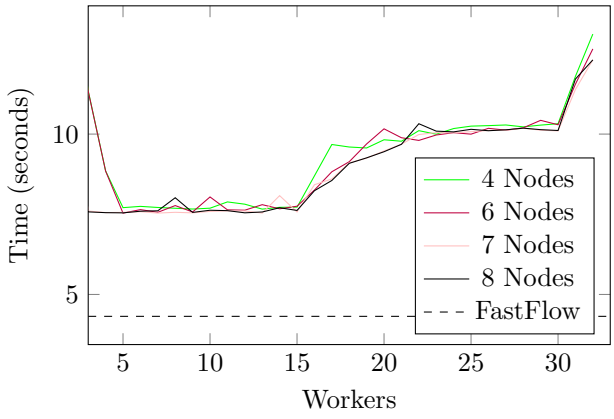


Figure 8: Implementation of a slave in MPI. The farm has been implemented using the architteture of Fast Flow. There is only one Left Worker which read the MPI messages coming from the master and split the work between the workers. After processing the data the Workers they transfer the data to the Gather. The gather after sends the data back to the Master using MPI.



(a) MPI: Time taken in seconds to **compress** 1 files of 2GB. The best time is with 8 Nodes and 15 Workers. The dashed line is the best time archived with Fast Flow.



(b) MPI: Time taken in seconds to **compress** 100 files of 20 MB each. The best time is with 6 Nodes and 5 Workers. The dashed line is the best time archived with Fast Flow.

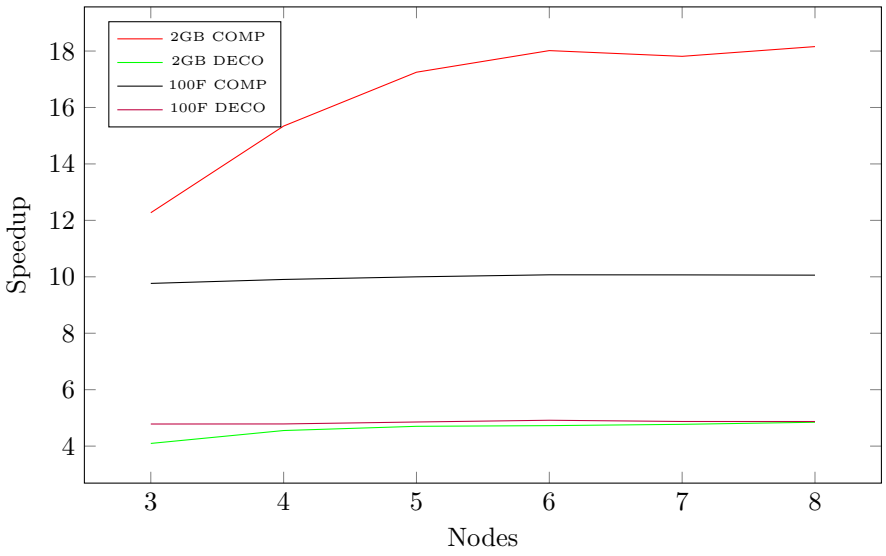


Figure 10: MPI: Speedup of the 2 datasets. The red and green line is the speedup of the compression and decompression of the 2GB dataset. The black and purple line is the speedup of the compression and decompression of the 100 Files of 20 MB dataset.

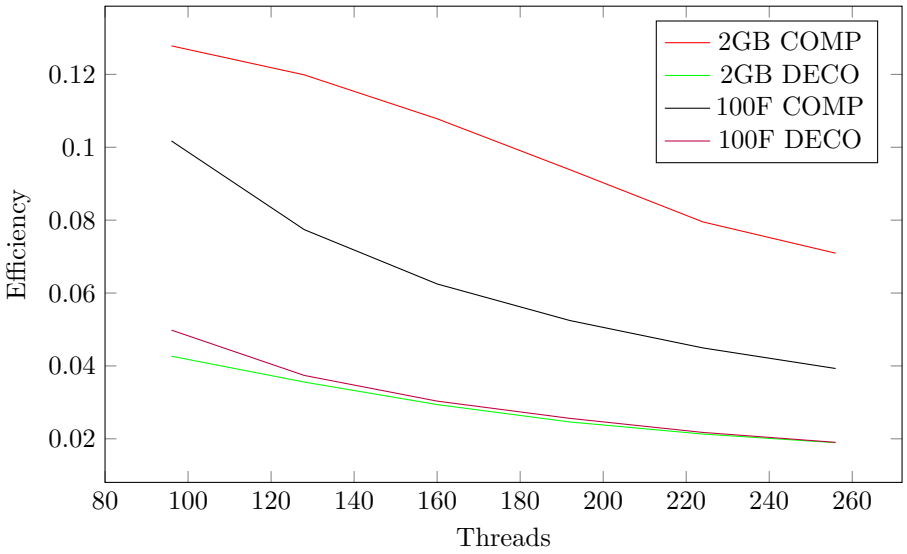


Figure 11: MPI: Efficiency of 2 datasets. The red and green line is the efficiency of the compression and decompression of the 2GB dataset. The black and purple line is the efficiency of the compression and decompression of the 100 Files of 20 MB dataset. To simplify the calculations every node uses all 32 Threads available to it.

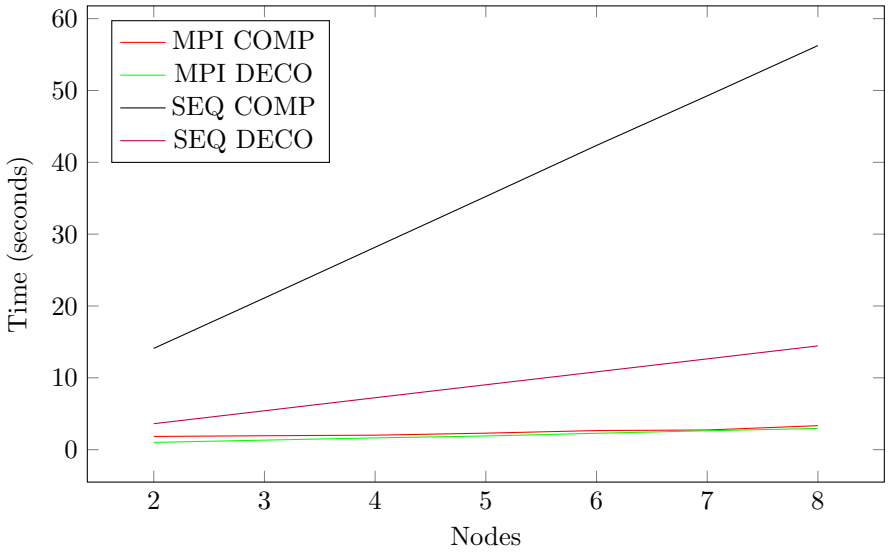


Figure 12: MPI: Weak Scaling. The dataset consist in a file of 200 MB * N where N is the number of nodes. Using the optimal number of workers for each node.

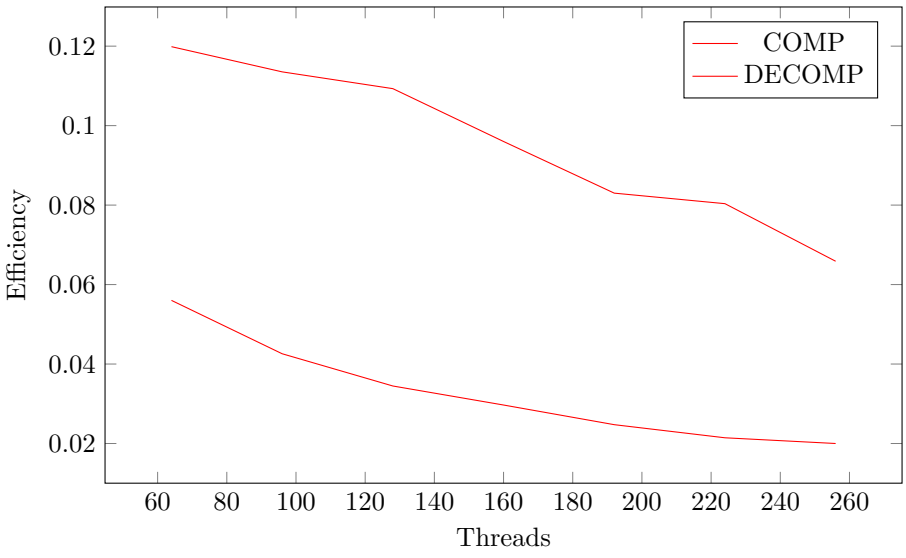


Figure 13: MPI: Weak Scaling Efficiency. The dataset consist in a file of 200 MB * $32N$ where N is the number of nodes. Using 32 Threads for each node.